

Práctica 1: Análisis léxico

OBJETO DE LA SESION

El objetivo de esta sesión es desarrollar un analizador léxico y un analizador sintáctico de acuerdo con la especificación que se presenta más adelante. Dichos analizadores recibirán como entrada un fichero de texto que contiene el programa fuente y en el caso de que no haya errores léxicos ni sintácticos producirán como resultado un código objeto que represente el programa fuente en forma de árbol. Si hay errores léxicos o sintácticos se generara mensajes de error para que se corrijan.

En este documento se presenta una descripción general del programa fuente que se utilizará en la práctica. Se debe notar que ciertas partes de dicha descripción no se van a utilizar en la práctica del analizador léxico. En particular, las comprobaciones relativas a los tipos de datos se realizarán más adelante (semántico).

DEFINICION DEL LENGUAJE DE PROGRAMACION A TRADUCIR

Descripción general del lenguaje de programación fuente PF2025

Empecemos con un pequeño programa de ejemplo, el cual constara de tres secciones principales encabezado, sección de declaración e inicialización de variables y cuerpo del programa:

```

pf2025 Ejemp1                                /* Encabezado, deberá llevar el nombre del programa*/

decl                                           /*Pal_Reserv Declaración de variables*/
/*Comentario linea*/
Ent pos#icion a,b,c;
cad d;
Bool e;

Inicio                                         /*Cuerpo del programa, inicio, {, etc*/
a:=5;
b:=3;
impcad("Dame un numero, Hola mundo");        /* solicita un número */
leerdig(var);                                  /* Guarda un valor entero en una variable */
d:=ab-c;
d:=a*(b-c);
impcad(d);
Fin                                           /*Cuerpo del programa, fin, }, etc*/

```

El programa en el lenguaje de programación X llamado PF2025 consta de las siguientes partes:

1. Palabra clave o reservada **pf2025** que indica el inicio del programa.
2. Un identificador (el nombre del programa **Ejemp1**).
3. *Opcionalmente*, pueden declararse variables que serán utilizadas dentro del programa. Las declaraciones de variables están formadas por un tipo de datos seguido de una lista de identificadores (nombres de variables). Los tipos de datos existentes en este lenguaje de programación son int, booleano y cadena.
Cuando se declara una variable se supondrá que inicialmente tomará el valor false si la variable es de tipo booleano, 0 si la variable es de tipo int y cadena vacía si la variable es de tipo cadena.
4. **Cuerpo del programa**, que consta de una secuencia de sentencias. Comienza con la palabra reservada **inicio** o un sinónimo y termina con **fin**, o sinónimo.

Las sentencias de un programa pueden ser de los siguientes tipos:

- Sentencia de asignación: Almacena en la variable indicada a la izquierda del símbolo **:=** el valor que se obtiene como resultado de evaluar la expresión a la derecha del símbolo **:=**.
- Sentencia condicional **if <Exp> then <StamentList> end.**
- Bucle **while <Exp> do <StamentList> end.** /* opcional*/
- Sentencia **leerdig**, esta sentencia lee un dato entero de entrada por teclado y lo almacena en una variable del mismo tipo.

Sentencia **impdig**, esta sentencia recibe como argumento una variable de tipo entero. El resultado de ejecutar esta sentencia es que imprime en pantalla dato tipo entero, como ejemplo el siguiente mensaje: El valor de la variable <variable> es: <x>

Donde <variable> es el nombre de la variable y <x> es el valor almacenado en dicha variable en el momento en que se ejecuta la sentencia **impdig**.

Nota: la comprobación de que la variable que se pasa como argumento a la sentencia impdig es de tipo int se realizará en la práctica 2.

- Sentencia **leercad**, esta sentencia lee un dato tipo cadena de entrada por teclado y lo almacena en una variable del mismo tipo.
- Sentencia **impcad**, esta sentencia recibe como argumento una variable de tipo cadena. El resultado de ejecutar esta sentencia es que imprime en pantalla el mensaje, ejemplo: “**Dame un numero**”.
- Sentencia **impBool**. Esta sentencia recibe como argumento una variable de tipo bool. El resultado de ejecutar esta sentencia es que se imprime por pantalla el siguiente mensaje:

El valor de la variable <variable> es: <x>

Donde <variable> es el nombre de la variable y <x> es el valor almacenado en dicha variable en el momento en que se ejecuta la sentencia **printBool**.

Nota: la comprobación de que la variable que se pasa como argumento a la sentencia printBool es de tipo bool se realizará en la práctica 2.

Especificación del analizador léxico

El analizador léxico a desarrollar debe reconocer los siguientes tokens:

- Palabras
 - reservadas: inicio (token INICIO), pf2025 (token PROG), decl (token DECL), impdig(token IMPDIG), impcad (token IMPCAD), leerdig(token LEERDIG), y end (token END).
- Las palabras clave para los tipos básicos: int, cad y booleano comparten el mismo token: TIPO.
- Signos de puntuación y operadores: ";" (token PC), "!=" (token ASIG), "+" (token MAS), "-" (token MENOS), "/" (token DIV), "*" (token MUL), "(" (token PAREN), ")" (token TESIS), "=" (token IGUAL), "," (token COMA).
- Identificadores (token id): comienzan por una letra (mayúscula o minúscula), seguida opcionalmente de cualquier cadena de letras y/o dígitos.
- Constantes, que pueden ser del siguiente tipo:
 - Números enteros no negativos (token CENT), es decir, cualquier cadena de dígitos. Ejemplos: 1, 0, 2010

El lenguaje **distingue mayúsculas y minúsculas**, las cuales deberán estar escritas en **español**; por lo tanto, mientras **inicio** es una palabra reservada, **Inicio** es un identificador e **INICIO** es otro identificador distinto del anterior.

Recuerde que también tiene que especificar, reconocer y consumir (sin generar token) los espacios en blanco, tabuladores, saltos de línea y retornos de carro.

Debe eliminar o ignorar los comentarios del programa fuente.

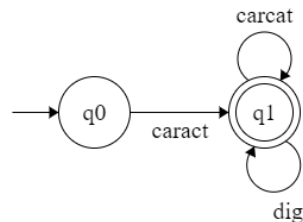
Debe generar mensajes de error en caso necesario guardando la información de la línea del programa fuente donde se encontró el error léxico.

Para desarrollar el analizador léxico pedido, hay que definir para cada token a reconocer, la **expresión regular** asociada y la acción asociada. A modo de ejemplo, se proporciona ya definido el token para el token **id**, recuerde que el **lexema** sería el nombre del identificador, ejemplo **posición** y el token se forma por dos campos bajo la estructura <nombre_token, atributo_token>. La regla para el patrón del token la puede recordar de la tabla siguiente:

TOKEN	DESCRIPCIÓN INFORMAL	LEXEMAS DE EJEMPLO
if	caracteres i, f	if
else	caracteres e, l, s, e	Else
comparacion	< o > o <= o >= o == o !=	<=, !=
id	letra seguida por letras y dígitos	pi, puntuacion, D2
numero	cualquier constante numérica	3.14159, 0, 6.02e23
literal	cualquier cosa excepto ", rodeada por "'s	"core dumped"

Figura 3.2: Ejemplos de tokens

Como puede observarse en la definición de la regla, la E.R para el `id` es la siguiente, *letra(letra|dig)**, y el AFD es como sigue:



Definición de la sintaxis del lenguaje de programación fuente

A continuación, vamos a dar la definición de la sintaxis del lenguaje fuente por medio de una gramática independiente del contexto (excepto para las expresiones, que se van a definir en lenguaje natural).

La gramática que define la sintaxis del lenguaje de programación fuente es la siguiente:

Tabla 3.1. Unidades sintácticas de un lenguaje ejemplo.

Palabras reservadas	Símbolos simples	Símbolos dobles	Otros
begin end bool int ref function if then fi else while do input output deref true false	; , + - * (= >	:= <=	número (uno o más dígitos) identificador (una letra seguida de 0 o más letras y/o dígitos)

```

<US> ::= <p_reservada> | <s_simple> | <s_doble> | <id> | <cte_num>
<p_reservada> ::= begin | end | bool | int | ref | function | if | then | fi | else | while
| do | repeat | input | output | deref | true | false
<s_simple> ::= ; | , | + | - | * | ( | ) | = | >
<s_doble> ::= := | <=
<cte_num> ::= <digito> | <cte_num> <digito>
<id> ::= <letra> | <letra><resto_id>
<resto_id> ::= <alfanumerico> | <alfanumerico><resto_id>
<alfanumerico> ::= <digito> | <letra>
<digito> ::= 0 | 1 | ... | 9
<letra> ::= a | b | ... | z | A | B | ... | Z

```

Expresiones en el lenguaje de programación fuente

Las expresiones del lenguaje fuente de programación deben de ajustarse a lo siguiente:

Expresiones sin tipo definido:

- Un identificador que sea el nombre de una variable. Será del tipo de la variable.
- Una expresión entre paréntesis. Será del tipo de la expresión contenida dentro de los paréntesis.

Expresiones de tipo ent:

- Una constante expresada en decimal.
- Una suma, resta, multiplicación o división de expresiones de tipo ent utilizando los operadores habituales "+", "-", "*", "/", ".".
- El opuesto de una expresión de tipo ent, utilizando el símbolo habitual "-" antes de la expresión.

Nota: para definir las expresiones puede ser necesario definir algún símbolo no terminal adicional en la gramática.

Reglas de precedencia

La precedencia de los operadores es, de menor a mayor:

1. union
2. intersec
3. in
4. or
5. and
6. not
7. =
8. < y >
9. + y -
10. * y /
11. - (opuesto)

Resumen

El prototipo del lenguaje de programación diseñado por el estudiante es el siguiente:

- 1.- Debe permitir la declaración de 3 tipos de datos
Entero, cadenas, booleanos.
- 2.- Debe permitir entradas y salidas
Leer (Enteros, cadenas y booleanos)
Imrpimir (Enteros, cadenas y booleanos)
- 3.- Decisiones If – else
- 4.- Identificar y manejar expresiones aritméticas con operadores como:
+, -, *, /, (,).
- 5.- Expresiones lógicas – relacionales.
>, <, ==, !=, and, or, not.
- 6.- Ciclos for, while (**opcional, por el momento**)

Documento y fichero a entregar

Se deberán entregar siguientes ficheros:

- Fichero `progfte.txt` que contiene el programa fuente, es un archivo de texto

- Fichero `progfte.dep` que contiene el pf depurado (eliminación de espacios en blanco, comentarios etc), también es un archivo de texto.
- Código comentado generado en el lenguaje que haya decidido utilizar C, Java, Python etc..
- Fichero o registro de la tabla de símbolos `progfte.tab`.
- Fichero que contiene la tabla de símbolos y lista de tokens `progfte.tok` también es un archivo de texto.

El documento a entregar deberá contener

- Portada con los datos comentados para los trabajos de entrega por unidad.
- Nombre del lenguaje
- Extensión por utilizar
- Explicar en su lenguaje cada instrucción, gramática, E.R y autómeta a utilizar así el cómo se realizará anotando la sintaxis y un ejemplo de su uso.

Evaluación

- 50% Programación de etapa léxico
- 20% Documentación y actividades complementarias
- 10% Examen teórico de la fase
- 10% Investigación de temas teóricos necesarios y explicación de la fase.
- 10% Entrega en tiempo y forma.